of times specified by `highChirp`. This will have the effect of sounding like a solid tone for as long as specified, in this case 20 milliseconds since `highChirp` is equal to 20.

While we have used the `delay()` function previously, albeit with little explanation, `delayMicroseconds()` is another type of delay that appropriately enough measures time in microseconds rather than milliseconds. This means that `delay(1)` and `delayMicroseconds(1000)` are equal to the same amount of time. Using our `chirpCycle` of 70%, `delayMicroseconds(chirpCycle * 10)` will delay for 700 microseconds after the pin is turned on and `delayMicroseconds(1000-(chirpCycle * 10))` will delay for 300 microseconds when the pin is turned off.

Next up, we need to turn off the speaker for either a longer delay midway through the chirp cycle, or for a short delay the rest of the time. The following two lines will check to see if we've hit the midway point or not:

```
digitalWrite(speakerPin, LOW);

 if (i == numChirps/2) delay(midChirp);
 else delay(lowChirp);
```

So far these lines of code have the effect of chirping five times in quick succession with only a slightly longer delay before chirping another five times. Once in a while it might be nice to toss things up a bit.

```
if ((random(chirpPause)) == 1) delay(random(200,1000));
    else delay(midChirp);
```

And these two lines do exactly that. As specified by `chirpPause`, we are providing a random chance of 1 in 8 that the cricket might pause for a breath of air that might last between 200 and 1000 milliseconds, or 0.2 up to 1 second in length. To accomplish this we are using the `random()` function that will be discussed in a later chapter.

```
delay(skittish);
```

Finally our last line of code in the `loop()` function is only ever reached when the PIR motion detector senses movement and sends a signal of `HIGH` to the Arduino. At this point, the cricket will wait for 5 seconds before checking to see if there is movement again. If someone remains too close to the cricket, it will stay quiet until they go away—and after running the cricket for any length of time I'm sure you will try to keep it quiet as much as you can.

Digital Functions

Now that we've seen the digital functions in action, let's look at each of them individually in greater detail to better understand how they work.

pinMode()

Before we can put the other digital functions to work, we first need to let the Arduino know how we plan to use its digital I/O pins. To do this, we use a function called `pinMode()` to set up which pin we are going to use and how we intend to use it. Its syntax is fairly simple, as follows:

```
pinMode(pin, state)
```